

PBL 02: 모바일 앱과 어택 서페이스(공격 표면)

정하은

1. 문제 정의

현대 사회에서 모바일 기기의 사용량은 폭발적으로 증가하고 있으며, 이는 단순한 사용 편의를 넘어 금융, 의료, 개인정보 등 민감한 데이터를 처리하는 주요 플랫폼으로 자리 잡고 있다. 이로 인해 데이터 유출, 소셜 엔지니어링, Wi-Fi 보안 사고와 같은 다양한 보안 위협이 급증하고 있다.

본 보고서는 모바일 앱을 네이티브 앱, 모바일 웹 앱, 하이브리드 앱 세 가지로 구분하고, 각 앱의 어택 서페이스(공격 표면)를 분석한 내용을 담고 있다. 또한, OWASP Mobile Top 10 2024를 기준으로 주요 보안 취약점을 식별하고, 안드로이드와 Apple의 플랫폼별 보안 가이드라인을 바탕으로 구체적인 대응 방안을 제시하는 것을 목표로 한다.

2. 모바일 앱의 종류 및 특징

모바일 앱은 개발 방식과 실행 환경에 따라 각기 다른 구조와 특징을 가지며, 이는 곧 고유한 공격 표면으로 이어진다.

네이티브 앱 (Native App)

- 정의 : 안드로이드(Java, Kotlin) 또는 iOS(Swift, Objective-C)와 같이 특정 운영체제(OS)의 언어로 개발된 앱이다.
- 특징 : OS에 최적화되어 최고의 성능과 안정성을 제공하며, 기기의 하드웨어 기능을 자유롭게 활용할 수 있다.

모바일 웹 앱 (Mobile Web App)

- 정의 : 모바일 브라우저에서 실행되는 웹 기반 애플리케이션이다.
- 특징 : 하나의 코드로 여러 OS를 지원할 수 있어 개발 비용이 저렴하고, 앱스토어 검수 없이 업데이트가 가능하다.

하이브리드 앱 (Hybrid App)

- 정의 : 네이티브 앱의 컨테이너(껍데기) 안에 웹 기술(HTML, CSS, JavaScript)로 만들어진 콘텐츠를 담아 실행하는 앱이다.
- 특징 : 하나의 코드베이스로 여러 플랫폼을 지원하면서도, 네이티브 기능 일부를 활용할 수 있어 개발 효율성이 높다.

특히, 안드로이드는 개방형 플랫폼으로 개발 자유도가 높고 리눅스 커널 기반의 샌드박스를 사용하며, iOS는 폐쇄형 플랫폼으로 보안성 강화에 중점을 두고 사용자 권한을 최소화하는 상반된 보안 정책을 가진다.

3. 모바일 앱의 공격 표면 및 주요 취약점

각 모바일 앱은 고유한 실행 환경 때문에 각기 다른 공격 표면을 가진다. OWASP Mobile Top 10 2024를 기준으로 주요 취약점을 분석했다.

앱 종류	주요 공격 표면	주요 취약점
네이티브 앱	• 단말기 내부: 루팅/탈옥된 기기, 앱 변조(리패키징), 바이너리 역분석 • 통신: 서버와의 통신 • 서버: 백엔드 API	M7: 불충분한 바이너리 보호: 난독화나 무결성 검사 없는 앱은 역분석 및 변조에 취약하다. M9: 안전하지 않은 데이터 저장: 키체인/키스토어 사용 없이 민감 정보를 저장하면 탈취될 수 있다. M8: 보안 구성 오류: 불필요한 권한 요청, 디버깅 기능 활성화 등 OS 설정을 잘못하면 공격에 노출된다.
모바일 웹 앱	• 웹 브라우저: 브라우저 샌드박스 취약점 • 통신: 웹 통신 • 서버: 웹 서버 및 백엔드 시스템	M4: 불충분한 입력/출력 유효성 검사: 웹 기반이므로 XSS, SQL 인젝션 등 웹 취약점에 그대로 노출된다. M5: 안전하지 않은 통신: HTTPS 미적용 시 통신 가로채기(Man-in-the-Middle) 공격에 취약하다. M3: 안전하지 않은 인증/인가: 인증/인가 로직이 서버에만 의존하여 클라이언트 조작에 취약하다.
하이브리드 앱	• 네이티브+웹: 두 환경의 모든 공격 표면 • 네이티브-웹 인터페이스	네이티브 앱과 모바일 웹 앱의 모든 취약점을 공유한다. M4: 불충분한 입력/출력 유효성 검사: 웹뷰를 통한 입력이 네이티브 코드에 영향을 줄 수 있다. M5: 안전하지 않은 통신: 웹뷰 통신과 네이티브 통신 모두에서 취약점이 발생할 수 있다.

특히, 모바일 환경의 보안 위협은 기기 임의 변경(루팅/탈옥), 네트워크 보안 위협(MITM), 그리고 물리적 위협(기기 분실/도난) 등 기존 PC 환경과 차별화된 특성을 가진다.

4. 플랫폼별 보안 강화 방안

모바일 앱의 공격 표면을 줄이기 위한 구체적인 대응 방안은 다음과 같이 플랫폼별로 구분하여 적용해야 한다.

4.1. 안드로이드 보안 강화 방안

안드로이드의 개방적 특성으로 인한 보안 위협에 대응하기 위해 다음과 같은 가이드라인을 준수해야 한다.

- **인증 및 사용자 데이터** : 인증 관리자(Authentication Manager)를 사용하고, Firebase 클라우드 메시징(FCM)을 통해 SMS 대신 IP 네트워킹을 사용한다.
- **데이터 보관** : 민감한 정보는 내부 저장소에 저장하고, 다른 앱과 공유할 때는 반드시 콘텐츠 제공자(Content Provider)를 사용해야 한다.
- **권한** : 앱에 필요한 최소한의 권한만 요청하고, signature 보호 수준의 권한을 우선적으로 고려해야 한다.
- **API 키** : API 키는 소스 코드에 직접 커밋하지 않고 Android 키 저장소를 사용하며, secrets-gradle-plugin을 활용해 안전하게 관리한다.
- **코드 보호** : 소스코드 난독화를 통해 앱의 역분석 및 변조를 방지해야 한다. ProGuard(무료)와 DexGuard(유료)와 같은 도구를 사용할 수 있다.

4.2. Apple 보안 강화 방안

iOS의 폐쇄적 특성을 활용하여 보안을 극대화하기 위해 Apple이 제공하는 기능을 활용해야 한다.

- **인증** : 사용자의 얼굴이나 지문 정보를 Secure Enclave에 저장하는 Face ID 및 Touch ID를 사용하고, Apple로 로그인과 패스키(Passkeys)를 통해 사용자의 프라이버시를 보호해야 한다.
- **안전한 통신** : 앱이 네트워크 통신을 할 때 ATS(App Transport Security)를 통해 TLS 1.2 이상을 강제하여 중간자 공격을 방지해야 한다.
- **사용자 데이터 보호** : 민감한 데이터는 키체인(Keychain)에 저장하여 보호하고, 앱의 권한은 앱 샌드박싱을 통해 해당 앱의 의도된 기능으로만 제한한다.
- **코드 안전성** : Developer ID 및 Gatekeeper를 통해 신원을 보증하고, 앱 공증을 받아 악성 코드 침투를 방지한다.

항목	Android 권장 방식	iOS 권장 방식
인증	FCM, Authentication Manager	Face ID, Apple 로그인, 패스키
데이터 저장	내부 저장소, Content Provider	키체인(Keychain)
통신 보안	HTTPS, networkSecurityConfig	ATS (TLS 1.2 이상)
코드 보호	ProGuard, DexGuard	Gatekeeper, 공증(App Notarization)

5. 결론 및 의견

네이티브, 모바일 웹, 하이브리드 앱은 각각의 장단점이 명확하며, 이로 인해 각기 다른 공격 표면을 가진다. 따라서 앱 개발자는 자신의 앱 종류와 특성을 정확히 이해하고, 이에 맞는 맞춤형 보안 전략을 수립해야 한다.

현대의 모바일 보안은 단순히 방화벽을 세우는 수준을 넘어, 개발 단계부터 보안을 고려하는 시큐어 코딩과 OS가 제공하는 강력한 보안 API를 적극적으로 활용하는 것이 필수적이다. 또한, MDM(Mobile Device Management)과 같은 솔루션을 도입하고, OWASP Mobile Top 10과 같은

최신 위협 동향을 지속적으로 파악하여 앱의 취약점을 선제적으로 진단하고 해결하는 능력이 중요하다. 궁극적으로 아무리 강력한 보안 시스템을 구축해도 사용자의 보안 의식이 부족하면 무용지물이 될 수 있다. 따라서 개발자는 앱의 보안을 강화하고, 사용자는 기본적인 보안 수칙을 준수하는 협력적인 노력이 필요하다고 생각한다.

참고 자료

1. OWASP. "Mobile Security Testing Guide v.1.4.0". OWASP Foundation, https://server.xcode.co.id/ebook8292/OWASP_MSTG-v1.4.0.pdf.
2. OWASP. "OWASP Mobile Top 10 2024". OWASP Foundation, <https://owasp.org/www-project-mobile-top-10/>.
3. Android. "Android 보안 가이드". Android Developers, <https://developer.android.com/privacy-and-security/security-tips?hl=ko>.
4. Apple. "iOS 보안". Apple Developer Documentation, <https://developer.apple.com/kr/security/>.
5. 이글루시큐리티. "모바일 보안위협과 대응방안". 이글루시큐리티 블로그, <https://www.igloo.co.kr/security-information/%EB%AA%A8%EB%B0%94%EC%9D%BC-%EB%B3%B4%EC%95%88%EC%9C%84%ED%98%91%EA%B3%BC-%EB%8C%80%EC%9D%91%EB%B0%A9%EC%95%88/>.